

Programmation concurrente et distribuée

1. Tâches.....	1
2. RwLock (verrou en lecture / écriture).....	1
3. Encodage / décodage UTF-8.....	2
4. Sérialisation / désérialisation (serde).....	2
5. Itération sur les lignes de l'entrée standard.....	2
6. Fichiers.....	3
7. Connecteurs UDP.....	3
8. Exemple de serveur TCP.....	3

1. Tâches

Lancer la fonction <code>do_something()</code> dans une tâche <code>my_task</code> .	<pre>my_task = tokio::spawn(async move { do_something().await; });</pre>
Attendre que la tâches <code>my_task</code> termine.	<pre>my_task.await?;</pre>

2. RwLock (verrou en lecture / écriture)

	<pre>use std::sync::Arc; use tokio::sync::RwLock;</pre>
Protéger la valeur <code>value</code> dans un verrou <code>my_lock</code> .	<pre>let my_lock = Arc::new(RwLock::new(value));</pre>
Acquérir le verrou en lecture sur <code>my_lock</code> et lire la référence protégée	<pre>let value = *my_lock.read().await;</pre>
Acquérir le verrou en écriture sur <code>my_lock</code> et écrire la référence protégée	<pre>*my_lock.write().await = ...;</pre>

0

3. Encodage / décodage UTF-8

Encoder une chaîne de caractères <code>my_message</code> selon l'encodage UTF-8 en tableau d'octet	<pre>let my_vec = my_message.as_bytes();</pre>
Décoder une chaîne de caractères <code>my_message</code> à partir de <code>nb_bytes</code> octets contenus au début d'un tableau <code>buf</code> de <code>u8</code> , selon l'encodage UTF-8	<pre>let my_messages = std::str::from_utf8(&buf[0..nb_bytes])?</pre>

4. Sérialisation / désérialisation (serde)

Sérialiser un objet <code>my_object</code> dans le JSON et selon l'encodage UTF-8.	<pre>let my_slice = serde_json::to_vec(&my_object)?;</pre>
Désérialiser un tableau d'octets <code>my_slice</code> au format JSON UTF-8 en un objet <code>my_object</code> de type <code>MyStructure</code> .	<pre>let my_object = serde_json::from_slice::<MyStructure>(&my_slice)?;</pre>

5. Itération sur les lignes de l'entrée standard

```
use tokio::io::{self, AsyncBufReadExt, BufReader};  
  
let mut lines = BufReader::new(io::stdin()).lines();  
while let Some(line) = lines.next_line().await? {  
    ...  
}
```

6. Fichiers

	<pre>use tokio::fs::File;</pre>
Créer un fichier fichier.txt	<pre>let my_file = File::create("fichier.txt").await?;</pre>
Ouvrir un fichier fichier.txt	<pre>let my_file = File::open("fichier.txt").await?;</pre>
Ecrire le vecteur d'octets my_slice dans le fichier my_file	<pre>my_file.write_all(&my_slice).await?; my_file.flush().await?;</pre>
Lire le contenu du fichier my_file dans le vecteur d'octets my_slice	<pre>let mut my_slice = vec![]; my_file.read_to_end(&mut my_slice).await?;</pre>
Supprimer fichier.txt	<pre>tokio::fs::remove_file("Fichier.txt").await?;</pre>

7. Connecteurs UDP

😊	<pre>use tokio::net::UdpSocket;</pre>
Créer un connecteur UDP qui écoute sur le port 12345	<pre>let my_socket = UdpSocket::bind("127.0.0.1:12345").await? ;</pre>
Créer un connecteur UDP qui écoute sur un port aléatoire	<pre>let my_socket = UdpSocket::bind("127.0.0.1:0").await?;</pre>
Envoyer depuis my_socket le vecteur d'octets my_vec à l'adresse 1.2.3.4 sur le port 12345	<pre>my_socket.send_to(my_vec, "1.2.3.4:12345").await?;</pre>
Recevoir un segment dans le tableau d'octets buf (qui est supposé alloué)	<pre>let (nb_bytes, sender_endpoint) = my_socket.recv_from(&mut buf).await?;</pre>

8. Exemple de serveur TCP

```
const LOCALHOST: &str = "127.0.0.1";
const PONG: &str = "PONG\n";

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let options = Options::parse();
    let listener = TcpListener::bind(format!("{:}", LOCALHOST,
option.port)).await?;
    loop {
        let (mut stream, _) = listener.accept().await?;
        tokio::spawn(async move {
            let (reader, mut writer) = stream.split();
            let mut lines = BufReader::new(reader).lines();
            if let Ok(Some(line)) = lines.next_line().await {
                writer.write_all(PONG.as_bytes()).await?;
                writer.flush().await?;
            }
            anyhow::Ok(())
        });
    }
}

let my_socket = UdpSocket::bind("127.0.0.1:0").await?;
```